Programming in Modern C++: Assignment Week 5

Total Marks : 18

Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology
Kharagpur – 721302
partha.p.das@gmail.com

August 14, 2025

Question 1

Consider the code segment below. *[MCQ, Marks 2]*

```
#include <iostream>
using namespace std;

class A {
    protected:
        int x;
    public:
        A(int val) : x(val) {}
        virtual void show() { cout << "A: " << x << endl; }
};

class B : public A {
    public:
        B(int val) : A(val + 1) {}
        void show() override { cout << "B: " << x << endl; }
};

int main() {
    A* aPtr = new B(5);
    aPtr->show();
    return 0;
}
```

What will be the output?

a) `A: 6`

b) `B: 6`

c) `A: 5`

d) `B: 5`

Answer: b)
Explanation:
The statement `A* aPtr = new B(5);` invokes the parameterized constructor of `B`, which forwards the call to the parameterized constructor of the base class with `val = val + 1 = 5 + 1 = 6`. Therefore, the data member `x` is initialized to 6.
Since `B` overrides `show()` (which is declared as `virtual`) and is called via a base pointer, dynamic dispatch calls `B::show()`, which prints `B: 6`.

Question 2

Consider the following code segment. *[MCQ, Marks 2]*

```
#include <iostream>
using namespace std;

class X {
    public:
        X() { cout << "X-ctor, "; }
        ~X() { cout << "X-dtor, "; }
};

class Y {
    X x;
    public:
        Y() { cout << "Y-ctor, "; }
        ~Y() { cout << "Y-dtor, "; }
};

int main() {
    Y y;
    return 0;
}
```

What is the output?

a) `X-ctor, Y-ctor, Y-dtor, X-dtor,`

b) `Y-ctor, X-ctor, Y-dtor, X-dtor,`

c) `X-ctor, Y-ctor, X-dtor, Y-dtor,`

d) `Y-ctor, X-ctor, X-dtor, Y-dtor,`

Answer: a)
Explanation:
Constructors are invoked in top-down order in the class hierarchy. Therefore, the statement `Y y;` results in executing the constructors in the following order: `X -> Y`. When main returns, the object `y` goes out of scope, invoking destructors. Destructors are invoked in bottom-up order in the class hierarchy. Therefore, the destructors are executed in the following order: `Y -> X`.

Question 3

Consider the following code segment. *[MCQ, Marks 2]*

```
#include <iostream>
using namespace std;

class Base {
    public:
        void fun() { cout << "Base::fun()"; }
};

class Derived : public Base {
    public:
        void fun() { cout << "Derived::fun()"; }
};

int main() {
    Derived d;
    --------------------
    return 0;
}
```

Fill in the blank so that the output is `Base::fun()`.

a) `d.fun();`

b) `Base::fun();`

c) `d.Base::fun();`

d) `Base::d.fun();`

Answer: c)
Explanation:
In this code, class `Derived` overrides the `fun()` method of class `Base`. However, based on the output given, the base class version of the function `fun()` must be invoked, which can be done as `d.Base::fun();`.

Question 4

Consider the following code segment. *[MCQ, Marks 2]*

```
#include <iostream>
using namespace std;

int data = 5;
class Base {
    protected:
        int data;
    public:
        Base() : data(10) {}
};

class Derived : Base {
    protected:
        int data;
    public:
        Derived() : data(15) {}
        void print() {
            cout << data << " " << Base::data << " " << ::data << endl;
        }
};

int main() {
    Derived d;
    d.print();
    return 0;
}
```

What will be the output?

a) `15 10 5`

b) `10 5 15`

c) `5 10 15`

d) `Compiler Error`

Answer: a)
Explanation:

- `data` inside `Derived` refers to its **own member**, initialized to `15`.
- `Base::data` refers to the **protected member** of `Base`, initialized to `10` in its constructor.
- `::data` refers to the **global variable**, value is `5`.

Question 5

Consider the following classes. *[MSQ, Marks 2]*

```
class A {
    public:
        void f1(int);
        void f2();
};

class B : public A {
    public:
        void f1();
        void f1(int);
        void f3();
};
```

Which statements is/are correct regarding the classes above?

a) `B::f1()` overrides `A::f1(int)`

b) `B::f1(int)` overrides `A::f1(int)`

c) `B::f1()` overloads `B::f1(int)`

d) `B::f3()` overrides `A::f2()`

Answer: b), c)
Explanation:

- **Function overriding** occurs when a derived class redefines a base class function with the **same signature**.
- So, `B::f1(int)` correctly **overrides** `A::f1(int)`.
- `B::f1()` is a **new function** with a different parameter list, so it **overloads** `B::f1(int)`. So, b) and c) are correct answer
- `B::f3()` is not related to `A::f2()`, so **(a) and (d) are incorrect**.

Question 6

Consider the following code segment. *[MCQ, Marks 2]*

```
#include <iostream>
using namespace std;

int gdata = 10;
class gmClass {
    int mdata;
    public:
        gmClass(int mdata_ = 0) : mdata(++mdata_) { ++gdata; }
        ~gmClass() { mdata = 0; gdata = 0; }
        void print() { cout << "mdata = " << mdata << ", gdata = " << gdata << endl; }
};

void fun() {
    gmClass ob;
    ob.print();
}

int main() {
    gmClass ob;
    fun();
    ob.print();
    return 0;
}
```

What will be the output?

a) mdata = 9, gdata = 11
   mdata = 1, gdata = 12

b) mdata = 9, gdata = 12
   mdata = 1, gdata = 0

c) mdata = 1, gdata = 11
   mdata = 2, gdata = 12

d) mdata = 1, gdata = 12
   mdata = 1, gdata = 0

Answer: d)
Explanation:

- Initially, `gdata = 10`.
- In `main()`, creating `ob` increments `gdata` to 11 and sets `mdata = 1`.
- In `fun()`, another object is created, setting `mdata = 1` and `gdata = 12`.
- On `fun()`'s end, destructor sets both `mdata` and global `gdata` to 0.
- Back in `main()`, printing `ob` shows `mdata = 1`, `gdata = 0`

Question 7

Consider the following program. *[MCQ, Marks 2]*

```
#include<iostream>
using namespace std;

class Account {
    int acc_no;
    string acc_type;
    double acc_balance;
    public:
        Account(int acc_no_, string acc_type_, double acc_balance_)
            : acc_no(acc_no_), acc_type(acc_type_), acc_balance(acc_balance_) {}
        void printStatus() {
            cout << acc_no << ":" << acc_type << ":" << acc_balance << ":";
        }
};

class Customer : private Account {
    int cust_id;
    string cust_name;
    public:
        Customer(int cust_id_, string cust_name_, int acc_no_,
            string acc_type_, double acc_balance_)
                : Account(acc_no_, acc_type_, acc_balance_),
        cust_id(cust_id_), cust_name(cust_name_) {}
        void printStatus() {
            Account::printStatus();
            cout << cust_id << ":" << cust_name;
        }
};

void print(Account* ac) {
    ac->printStatus(); // LINE-1
}

int main() {
    Customer* cObj = new Customer(101, "Rajiv", 2002023, "current", 20000.00);
    print(cObj); // LINE-2
    return 0;
}
```

What will be the output/error?

a) `2002023:current:20000:`

b) `2002023:current:20000:101:Rajiv`

c) `Compiler error at LINE-1: Account::printStatus() is inaccessible in function 'print'`

d) `Compiler error at LINE-2: Account is an inaccessible base of Customer`

Answer: d)
Explanation:
In `LINE-2`, while calling `print(cObj);` private inheritance prevents pointer conversion from `Customer*` to `Account*`. Hence `LINE-2` causes error.

Question 8

Consider the following program. *[MCQ, Marks 2]*

```
#include<iostream>
using namespace std;

class PCP {
    public:
        PCP() {}
        ~PCP() {}
    private:
        PCP(const PCP& obj) {}
        PCP& operator=(const PCP&) { return *this; }
};

class IntData : public PCP {
    int i;
    public:
        IntData() {}
        IntData(const int& i_) : i(i_) {}
        void print() { cout << i << " "; }
};

int main() {
    IntData i01(10);
    IntData i02(20);
    i01 = i02;
    i01.print();
    i02.print();
    return 0;
}
```

What will be the output/error?

a) `20 10`

b) `20 20`

c) `Compiler error: operator= is private`

d) `Compiler error: copy constructor is private`

Answer: c)
Explanation:

- Class `PCP` has a **private copy constructor** and **private assignment operator** to prevent copying or assignment.
- `IntData` publicly inherits from `PCP`. When `i01 = i02` is executed, it tries to call `PCP::operator=`, but it's **inaccessible** due to being `private`.
- Hence, the compiler throws an **error**, and the program **fails to compile**.

Question 9

Consider the following program. *[MCQ, Marks 2]*

```
#include<iostream>
using namespace std;

class A {
    public:
        int a;
};

class B : protected A {
    public:
        int b;
};

class C : public B {
    public:
        int c;
        C(int a_, int b_, int c_) {
            a = a_; // LINE-1
            b = b_;
            c = c_;
        }
};

int main() {
    C cObj(10, 20, 30);
    cout << cObj.a << " "; // LINE-2
    cout << cObj.b << " "; // LINE-3
    cout << cObj.c;
    return 0;
}
```

What will be the output/error?

a) `10 20 30`

b) `Compiler error at LINE-1: a is not accessible`

c) `Compiler error at LINE-2: a is not accessible`

d) `Compiler error at LINE-3: b is not accessible`

Answer: c)
Explanation:
`a` is protected in `class B`, and stays protected in `class C`; hence it is inaccessible outside class `C`, i.e. in `main()`. So, Line-2 throws the error.

Programming Questions

Question 1

Consider the following program. Fill in the blanks as per the instructions given below:

- at `LINE-1` with appropriate initialization list to initialize the data members,
- at `LINE-2` to call `printContact()`,

such that it will satisfy the given test cases. *Marks: 3*

```
#include<iostream>
using namespace std;

class Contact{
    private:
        int phone_no;
    public:
        Contact(int phone_no_) : phone_no(phone_no_){}
        void printContact(){
            cout << "phone: " << phone_no << endl;
        }
};

class Student : private Contact{
    private:
        int roll_no;
        string name;
    public:
        Student(int roll_no_, string name_, int phone_no_)
                        : ________________________ {}    //LINE-1
        ________________________;                        //LINE-2
        void print(){
            cout << "roll: " << roll_no << endl;
            cout << "name: " << name << endl;
        }
};

int main(){
    int roll, pnum;
    string name;
    cin >> name;
    cin >> roll >> pnum;

    Student s(roll, name, pnum);
    s.print();
    s.printContact();
    return 0;
}
```

Public 1

```
Input: Himadri 100 9812345678
```

```
Output:
roll: 100
name: Himadri
phone: 2147483647
```

Public 2

```
Input: Soumen 200 9312345678
Output:
roll: 200
name: Soumen
phone: 2147483647
```

Private

```
Input: Arup 300 9912345678
Output:
roll: 300
name: Arup
phone: 2147483647
```

Answer:
`LINE-1: roll_no(roll_no_), name(name_), Contact(phone_no_)`
`LINE-2: using Contact::printContact`
Explanation:
At `LINE-1` the initialization list to initialize the data members and to invoke base class constructor can be done as:

```
Student(int roll_no_, string name_, int phone_no_)
                    : roll_no(roll_no_), name(name_), Contact(phone_no_)}
```

Due to private inveritance, at `LINE-2` we need to add the following line of code to make `printContact` accessible through `Student` object.
`using Contact::printContact`

Question 2

Consider the following program. Fill in the blanks as per the instructions given below:

- At `LINE-1` and `LINE-3` fill in the blanks to inherit class `Vehicle`;
- At `LINE-2` and `LINE-4` fill in the blanks with proper initialization lists;

such that it satisfies the given test cases. *Marks: 3*

```
#include<iostream>
using namespace std;

class Vehicle{
    protected:
        int no_of_wheels;
    public:
        Vehicle(int nw) : no_of_wheels(nw){}
        friend ostream& operator<<(ostream& os, const Vehicle& d);
};

class Car : _________________ {    //LINE-1
    protected:
        int no_of_passengers;
    public:
        Car(int nw, int np) : ____________ {} //LINE-2
        friend ostream& operator<<(ostream& os, const Car& d);
};

class Truck : ____________ {    //LINE-3
    protected:
        int load_capacity;
    public:
        Truck(int nw, int lc) : __________ {} //LINE-4
        friend ostream& operator<<(ostream& os, const Truck& d);
};

ostream& operator<<(ostream& os, const Vehicle& ob) {
    os << "no_of_wheels: " << ob.no_of_wheels << endl;
    return os;
}

ostream& operator<<(ostream& os, const Car& ob) {
    os << "no_of_wheels: " << ob.no_of_wheels << ", no_of_passengers: "
       << ob.no_of_passengers << endl;
    return os;
}

ostream& operator<<(ostream& os, const Truck& ob) {
    os << "no_of_wheels: " << ob.no_of_wheels << ", load_capacity: "
       << ob.load_capacity << endl;
    return os;
}
```

```
int main(){
    int wheelCount, carWhell, passengers, truckWheel, load;
    cin >> wheelCount >> carWhell >> passengers >> truckWheel >> load;
    Vehicle v(wheelCount);
    Car c(carWhell, passengers);
    Truck t(truckWheel, load);
    cout << v << c << t;
    return 0;
}
```

Public 1

```
Input: 2 4 6 8 30
Output:
no_of_wheels: 2
no_of_wheels: 4, no_of_passengers: 6
no_of_wheels: 8, load_capacity: 30
```

Public 2

```
Input: 2 4 7 10 50
Output:
no_of_wheels: 2
no_of_wheels: 4, no_of_passengers: 7
no_of_wheels: 10, load_capacity: 50
```

Private

```
Input: 2 3 4 8 20
Output:
no_of_wheels: 2
no_of_wheels: 3, no_of_passengers: 4
no_of_wheels: 8, load_capacity: 20
```

Answer:

`LINE-1:` `public Vehicle`
`LINE-2:` `no_of_passengers(np), Vehicle(nw)`
`LINE-3:` `public Vehicle`
`LINE-4:` `Vehicle(nw), load_capacity(lc)`

Explanation: Both `Car` and `Truck` **inherit** from `Vehicle`, which can be implemented as:

```
class Car :  public Vehicle { //LINE-1
class Truck :  public Vehicle { //LINE-3
```

In their constructors (`LINE-2` and `LINE-4`), we **call the base class constructor** `Vehicle(nw)` to initialize `no_of_wheels`.

Their own members (`no_of_passengers`, `load_capacity`) are initialized in the member initializer list. This ensures a proper construction and allows the friend `operator<<` functions to access `protected` members.

Question 3

Consider the following program. Fill in the blanks according to the instructions given in the following: *Marks: 3*

- At `LINE-1`, `LINE-2` and `LINE-3` with appropriate initialization lists;
- At `LINE-4`, `LINE-5` and `LINE-6` with appropriate return statements;

such that it satisfies the given test cases.

```
#include<iostream>
using namespace std;

class A{
    int a;
    public:
        A(int _a = 0);
        int sum();
};

class B : public A{
    int b;
    public:
        B(int _a = 0, int _b = 0);
        int sum();
};

class C : public B{
    int c;
    public:
        C(int _a = 0, int _b = 0, int _c = 0);
        int sum();
};

A::A(int _a) : _______ {}    //LINE-1
B::B(int _a, int _b) : _______ {}    //LINE-2
C::C(int _a, int _b, int _c) : _______ {}    //LINE-3
int A::sum(){ _____________; }    //LINE-4
int B::sum(){ _____________; }    //LINE-5
int C::sum(){ _____________; }    //LINE-6

int main(){
    int a, b, c;
    cin >> a >> b >> c;
    A aObj(a);
    B bObj(a, b);
    C cObj(a, b, c);
    cout << aObj.sum() << ", " << bObj.sum() << ", " << cObj.sum();
    return 0;
}
```

Public 1

```
Input: 10 20 30
```

```
Output: 10, 30, 60
```

Public 2

```
Input: 30 20 40
Output: 30, 50, 90
```

Private

```
Input: 30 40 40
Output: 30, 70, 110
```

Answer:

```
LINE-1:  a(_a)
LINE-2:  A(_a), b(_b)
LINE-3:  B(_a, _b), c(_c)
LINE-4:  return a
LINE-5:  return A::sum() + b
LINE-6:  return B::sum() + c
```

Explanation: At `LINE-1`, `LINE-2` and `LINE-3` with appropriate initialization lists for initializing data members and base class members can be written as follows:

```
A::A(int _a) :  a(_a) {}    //LINE-1
B::B(int _a, int _b) : A(_a), b(_b) {}    //LINE-2
C::C(int _a, int _b, int _c) : B(_a, _b), c(_c) {}    //LINE-3
```

The `return` statements at `LINE-4`, `LINE-5` and `LINE-6` to satisfy the test cases can be written as:

```
int A::sum(){ return a; }    //LINE-4
int B::sum(){ return A::sum() + b; }    //LINE-5
int C::sum(){ return B::sum() + c; }    //LINE-6
```